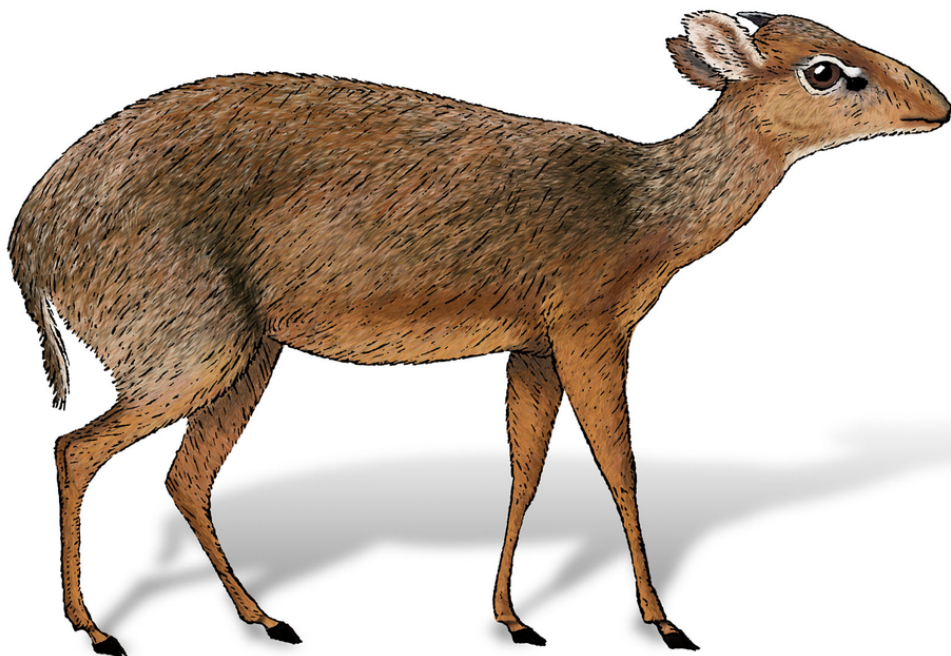


O'REILLY®

Learning DevSecOps

A Practical Guide to Processes and Tools



Steve Suehring

Learning DevSecOps

A Practical Guide to Processes and Tools

Steve Suehring

Learning DevSecOps

by Steve Suehring

Copyright © 2024 Steve Suehring. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

- Acquisitions Editor: Simina Calin
- Development Editor: Melissa Potter
- Production Editors: Jonathon Owen and Clare Laylock
- Copyeditor: nSight, Inc.
- Proofreader: Piper Editorial Consulting, LLC
- Indexer: nSight, Inc.
- Interior Designer: David Futato
- Cover Designer: Karen Montgomery
- Illustrator: Kate Dullea
- May 2024: First Edition

Revision History for the First Edition

- 2024-05-15: First Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781098144869> for release details.

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Learning DevSecOps*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the author, and do not represent the publisher's views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-098-14486-9

[LSI]

Preface

DevSecOps jobs are abundant, but looking at the requirements for those jobs, it's quickly evident that there is no agreement on what DevSecOps actually entails. That's what made this book quite difficult to write. I've written books on everything from MySQL to JavaScript to Windows Server to Linux Firewalls. Each of those technologies has a well-defined scope. Writing on Linux firewalls does not require covering several different technologies and skills in the same book. But DevSecOps is not as well-defined. Writing on DevSecOps exposes the fissures in how we define technologies, between the actual hands-on work and the hype. Even the term "DevSecOps" is not as widely used as the term "DevOps." Granted, "DevSecOps" does not roll off the tongue as easily as "DevOps," but it's more than that. Simply lending a voice to the definition of DevSecOps is one of the reasons that I wrote this book.

The goal of this book is not to be a comprehensive step-by-step guide to implementing DevSecOps, whatever the term means. That book is impossible to write because of the rapid changes in tools and the highly customized needs of each organization moving toward DevSecOps. Rather, the goal of this book is to provide patterns of success while also exposing some of the technologies and practices involved in large DevSecOps deployments. The book does not cover every software tool that an organization might use in DevSecOps. This is not an omission, or if it is, the omission is intentional so that the focus can remain on processes and people rather than technology and tools. Tech and tooling will change, but having the best people implementing the best processes will always work.

What Is DevSecOps?

What is DevSecOps? It depends on who you ask. As defined in this book, DevSecOps is a set of agile and iterative practices that help to deliver software and technology systems rapidly, accurately, and repeatedly, emphasizing processes and people above tools.

DevSecOps is about culture first. I've worked for organizations that were so far away from agile and iterative as to be spinning backward in the software development lifecycle. In such organizations, technologies are chosen by unqualified people without any consideration for workflow, productivity, or best practices, much less the end user. Deadlines are chosen before we even know what we're building. Contrast, then, DevSecOps cultures, where testing and security are natural extensions within the development process rather than additions later on. Automation and scripting are heavily emphasized in DevOps and DevSecOps.

Computing has a great way of reinventing itself over and over again. Many of the practices shown in this book have been around since the early days of computing. Mainframes allowed for slices of computing time and resources, and that's what we do today with cloud provisioning, just on a grander scale. Many of the things we do today as modern DevSecOps practices have been around for decades on Linux. Scripting and automation is not new, but formalizing it and getting buy-in from everyone involved in an organization is the value brought by DevSecOps. That's the essence of DevSecOps: enabling people to use processes and tools to rapidly and repeatedly improve the quality of software.

Who Is This Book For?

This book is for anyone interested in learning about DevSecOps and its predecessor, DevOps. You might be involved in development, operations, or security and want to learn about the melding of all three into a set of tools and processes for making production-level deployments easier. To get maximum value from the entire book, you should have a computing background, but everyone interested in DevSecOps will benefit from [Chapter 1](#), even those without a computing background.

Being able to write code, commit and push the code, and have tests automatically executed on that code is one such practice in DevSecOps. Scaling across multiple cloud providers is common as well. All of this is done seamlessly. Of course, all of that automation needs people who understand not only the goals of the automation but how to configure it. With that in mind, if you're interested in learning about the processes involved in DevSecOps while also being exposed to some of the technologies involved, then this book should be helpful.

How This Book Is Organized

This book is organized into eight chapters. With a few exceptions, the chapters are largely standalone, meaning that you can read only the chapters or sections of chapters that you find valuable.

Chapter 1, “The Need for DevSecOps”, helps to frame the story for the rest of the book. The chapter demonstrates how software was developed with methodologies like Waterfall and Agile and how software is developed with DevSecOps. **Chapter 1** also discusses the need to tear down departmental silos and places an emphasis on the importance of culture in DevSecOps.

Chapter 2, “Foundational Knowledge in 25 Pages or Less”, condenses some of the most basic knowledge needed to be successful at DevSecOps—or at least lays the foundation for gaining such knowledge. If there was a way to condense weeks’ worth of computing course material into one place, this is hopefully that place.

Chapter 3, “Integrating Security”, continues with some of the foundational material from **Chapter 2** but with a focus on security. Building on that foundation, we’ll discuss the OWASP ZAP tool in **Chapter 3**.

Chapter 4, “Managing Code and Testing”, looks at git and the Gitflow pattern in DevSecOps. The chapter also covers the various levels of testing.

Chapter 5, “Moving Toward Deployment”, introduces management of configuration as code along with Docker. We’ll also build a local registry for Docker.

Chapter 6, “Deploy, Operate, and Monitor”, examines Ansible and Jenkins for deployment and code building. Both of these technologies are widely used, though they’re certainly not the only technologies that perform deployment and build tasks. **Chapter 6** also discusses monitoring, with an emphasis on best practices for monitoring.

Chapter 7, “Plan and Expand”, integrates Kubernetes into the DevSecOps coverage, clustering and expanding the deployment of software in an organization.

Chapter 8, “Beyond DevSecOps”, wraps up the book with coverage of five patterns and takeaways from successful DevSecOps organizations.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

`Constant width`

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

Constant width italic

Shows text that should be replaced with user-supplied values or by values determined by context.

TIP

This element signifies a tip or suggestion.

NOTE

This element signifies a general note.

WARNING

This element indicates a warning or caution.

O'Reilly Online Learning

NOTE

For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit <https://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.

1005 Gravenstein Highway North

Sebastopol, CA 95472

800-889-8969 (in the United States or Canada)

707-827-7019 (international or local)

707-829-0104 (fax)

support@oreilly.com

<https://www.oreilly.com/about/contact.html>

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at <https://oreil.ly/LearningDevSecOps>.

For news and information about our books and courses, visit <https://oreilly.com>.

Find us on LinkedIn: <https://linkedin.com/company/oreilly-media>.

Watch us on YouTube: <https://youtube.com/oreillymedia>.

Acknowledgments

Thank you to the technical reviewers, Patrick Dubois, David Volm, Swapnil Shevate, and Vladislav Bilay, for their time, effort, and feedback. They helped identify areas that needed additional coverage and gave their expertise in other areas of the book. Thanks to Rob, Jim, and Jaclyn at Partners and to Tim for the assistance with review questions.

Chapter 1. The Need for DevSecOps

Software is created to solve problems. However, too often, creating software comes with its own set of problems, sometimes even creating new problems along the way. An organization makes a decision whether to develop customized software or to purchase prebuilt software. The prebuilt option is most economical for commodity software like an office productivity suite. But custom development is often needed for development of advanced solutions in business functional areas. Custom solutions are created in pursuit of the ultimate goals of gaining competitive advantage or increasing efficiency.

The process of developing software changed significantly in the late 1990s and into the early 2000s. That major shift went from an intense focus on gathering requirements to a focus on iteration and speed. The iterative manner in which software is developed features repeatable processes and automation that enable rapid delivery of new features, incorporating feedback loops throughout the development lifecycle. Together with organizational cultural changes that promote an open source, transparent mentality, the result is cross-functional teams concerned more with quality than territory and merging of multiple teams: Development, Operations, and Security—DevSecOps.

This chapter looks at the drivers behind the DevSecOps movement. The process of software development is the initial focus. The evolution of software development methodologies provides the background needed to fully understand, and thus be successful at, DevSecOps. The chapter continues with an emphasis on the importance of cultural changes for organizations moving toward DevSecOps.

Developing Software

To achieve their goals, organizations allocate some of their resources to create software. It's important to consider that these resources could be invested elsewhere where the resources might gain a higher return. For example, investing \$100,000 into marketing might result in more customers than investing those funds into streamlining the customer sign-up process on the website.

Even if money is not a concern, speed is. The ability to create and then deploy software quickly is a limiting factor on any effort to gain competitive advantage or increase efficiency. After a certain point, adding more developers to a project does not get that project done any faster. Just the opposite. As more developers are added, coherent communication becomes impossible.

Software starts as an idea. Taking that idea and turning it into working software requires forethought and planning. A software development project can be managed using several processes, depending in part on the type of software being developed. Software development involves defining the requirements, designing the solution, developing and coding, and finally testing the software just prior to release. This process is illustrated in **Figure 1-1**.

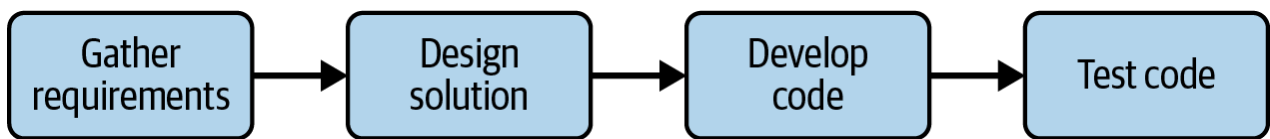


Figure 1-1. A process for software development

The four stages, sometimes called a software development lifecycle (SDLC), can be conceptualized as a waterfall, with each stage producing one or more artifacts, which are then passed or fall to the next stage, more like **Figure 1-2**.

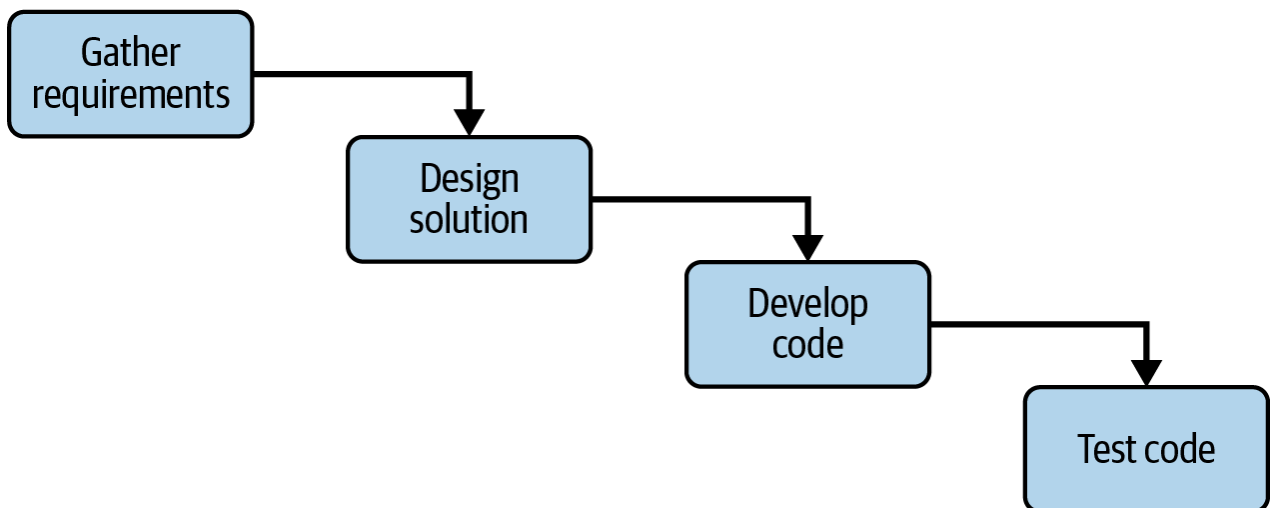


Figure 1-2. Completing each phase of a project in waterfall development

When using a methodology like waterfall to create software, each stage is completed prior to moving on to the next stage. This is illustrated within **Figure 1-1** where requirements are gathered and documented before moving on to the design phase, labeled “Design solution” in **Figure 1-1**. If a new

requirement is discovered during the design phase or additional questions lead to new requirements, those elements are frequently added into a follow-on project.

At the end of the requirements-gathering phase, the project formally has a scope defined, which includes all of the features of the software. These features incorporate the primary functions of the software along with additional features that aren't technically required for the software to function but are expected. These nonfunctional requirements are items like responsiveness or speed, security, and other behaviors of the application. Without capturing and adding the nonfunctional requirements, the resulting software product will leave users frustrated and underwhelmed.

Consider a business requirement: enabling a customer to find a product and place an order. Prior to computers, this business requirement was fulfilled in any number of ways, including the customer walking into a store, finding, and then purchasing the product or using a catalog to find the product, calling the company, and placing the order via telephone. With computers and the internet, this business requirement is now frequently accomplished through the web.

Fulfilling the business requirement of enabling a customer to find a product and place an order using a website leaves significant space to find a solution. Uploading a PDF of the catalog to the website and providing a form that enables the customer to email their order fulfills the minimal functional requirements for the site. However, even though the requirement is fulfilled, most users would expect something different and probably wouldn't order with such a clumsy process that lacks many of the features that customers take for granted within the user experience of ordering products online.

Instead, nonfunctional requirements also need to be captured. A few exploratory questions to the stakeholder or project sponsor would reveal rich detail about the intent for the solution. For example:

- How will products be represented on the site (photos, narrative, technical specifications, and so on)?
- Who will take product photos and produce them for the web, and who will write the narrative production description?
- How will inventory be updated so that customers can't order products that are out of stock?
- How will orders be placed?
- How will employees be alerted when a new order is placed?
- Who will maintain the online catalog with new products?
- What forms of payment are accepted?
- Do customers need to create accounts, track order history, track shipping?

These questions represent just a small fraction of the questions that would need to be answered during an initial exploratory or feasibility meeting. Some of these questions are already or will quickly become

functional requirements during the feasibility phase or during the requirements-gathering phase. However, absent someone in the meeting who has deployed a project such as this, there would surely be missed requirements.

The scope of the project, then, defines those elements that are included and delineates other elements that are not meant to be included within a project. Anything not specifically included is assumed to be excluded and thus out of scope for the project. If a fundamental requirement was missed, the project sponsor will face the unhappy choice of redefining scope or moving forward without that requirement and then adding the missed feature in a later follow-up project.

Months or even years of calendar time can elapse between the idea and the implementation. The delay between idea and released software product makes waiting for a missed feature even more painful for the project sponsor. Within that delay, any competitive advantage that might have been realized can quickly evaporate when a competitor who didn't miss the requirement releases their own version.

The following sections examine some of the problems and associated solutions surrounding modern software development.

Developing Agility

In response to the lag between project definition and completion, organizations have turned toward iterative processes like Agile and Scrum as a means to rapidly deliver value to the stakeholder. With an iterative software development process, all four stages described earlier (requirements, design, development, and testing) are performed. Rather than attempting to capture all requirements for all possible aspects of the project, iterative development focuses on the features that are of the highest value to the stakeholder. The highest-value features are then expanded through a round of requirements gathering before being designed, developed, tested, and released, with a short cycle of two to four weeks. **Figure 1-3** shows how each phase of the SDLC is handled with an iterative process like Agile.

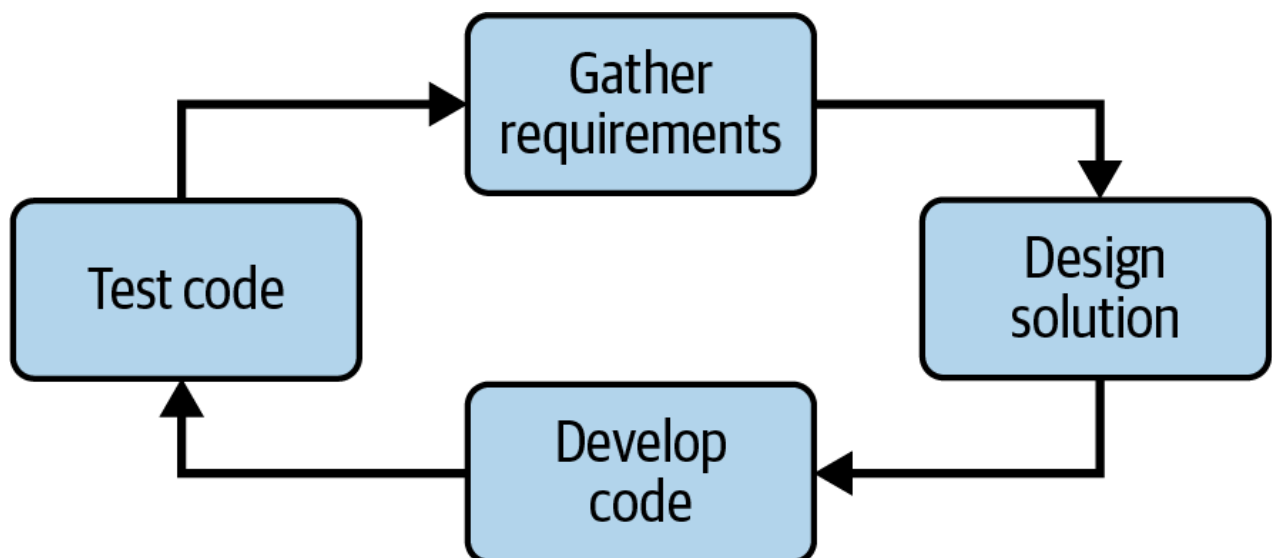


Figure 1-3. Iterating through each phase and then starting over with an Agile-like process

As illustrated in **Figure 1-3**, each phase is completed, but there is no attempt to gather full requirements because of the learning process associated with iterative development. If a requirement is missed, the stakeholder can choose to not release the feature or add the missed requirement in the next iteration. With an iterative development process, the next release is only weeks away rather than months or years away. Contrast that to a missed requirement in a waterfall process, where the next release may be months or years away, and you can see the clear benefit of this process.

Iterative development also enables rapid response to changing market conditions. For example, you might have the best idea for the next killer app, start development on that app, but then have your competitor release essentially the same app. In a waterfall model, you would need to scrap the project entirely. With an iterative process, focus can be shifted toward features that might be missing from the competitor's app.

Agile software development features several ceremonies such as sprint planning, daily stand-up, sprint review, sprint retrospective, and backlog grooming. An overall backlog or list of all of the possible features known at a given moment is created and prioritized. From that prioritized list of features, a sprint backlog is created. The sprint backlog is a commitment from the development team of which features will be implemented during the current iteration. The sprint backlog is created based on availability of team members and their estimation of effort, also called level of effort (LOE), for each individual item on the backlog.

At the end of the sprint, a sprint review is conducted where the team shows off what it has accomplished during that iteration. After the sprint review has been completed, the team examines what might have been done differently during the sprint within the retrospective. A team might answer three questions during the retrospective:

- What should we start doing?
- What should we stop doing?
- What should we continue doing?

These three questions enable the team to reflect on what worked, what didn't work, and what they might change moving into the next iteration. With the retrospective complete, the team can move toward backlog grooming, where the product backlog is refined and reprioritized. The stakeholder or product owner is usually involved in the backlog refinement process to set priority for the team.

Developing Broken Software

Flawed requirements lead to flawed software, or software that doesn't meet the original requirement. Flawed software can happen regardless of whether that original requirement was successfully elicited from the project sponsor. The end result is dissatisfaction, broken functionality, and security problems.

When examining the requirements, developers are often left with questions. These questions range from the mundane, such as where to place the curly braces for a conditional in some languages, to the critical, such as obtaining credentials for a database connection. In the latter case, development may need to stop while those credentials are obtained. In other cases, developers simply answer the question to the best of their ability and keep moving forward.

Developing software in a silo, devoid of interaction with anyone other than developers, leads to broken software. In the siloed development style, using a waterfall or similar methodology, the developers examine and interpret requirements to the best of their ability. Consider the following question: “In which web browsers should the site work?” along with a common answer: “Browsers? I’ve been developing using Chrome; I didn’t think about the site working in other browsers.” **Figure 1-4** illustrates development in a silo, where developers, operations staff, and security engineers don’t communicate well.

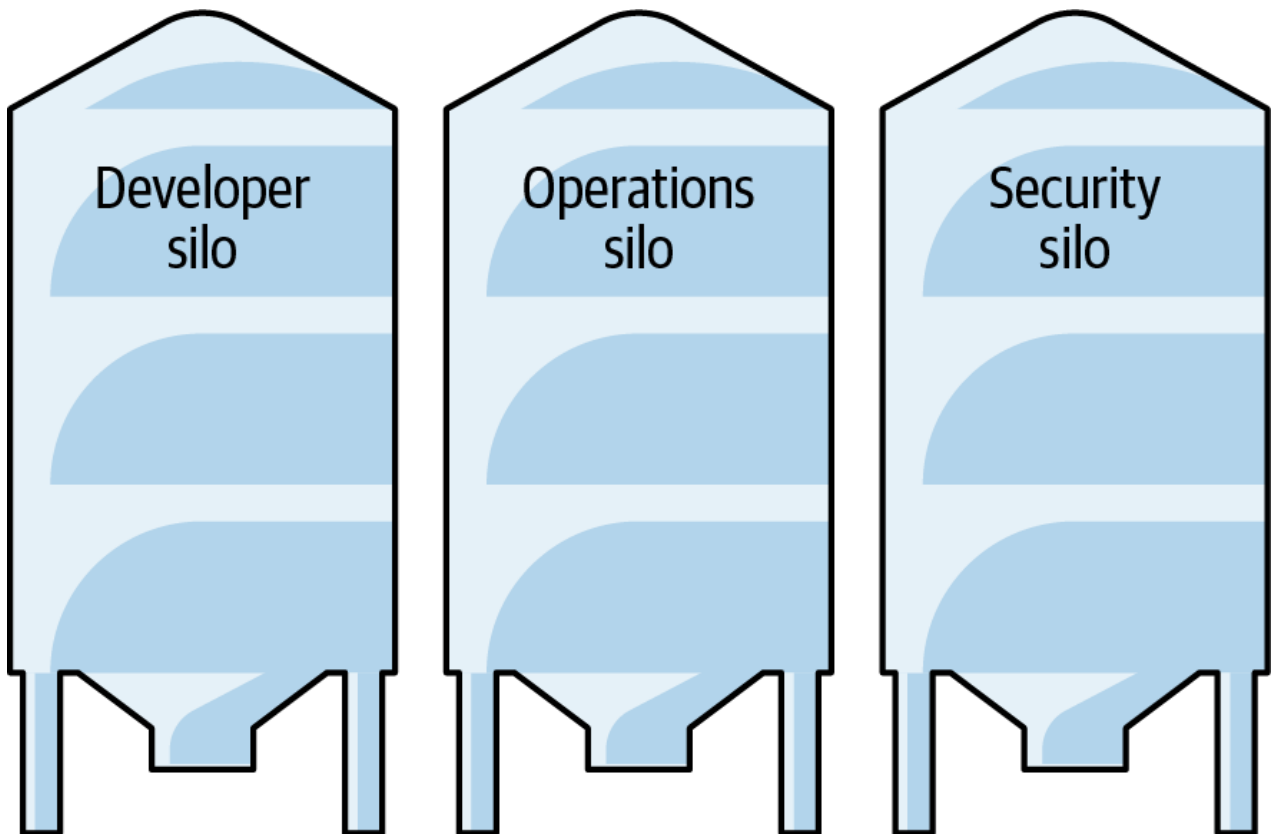


Figure 1-4. Siloed development in an organization leads to lack of visibility

Deadlines dictate the number of features and the quality of those features. The deadline for delivery may be such that there is no time to even identify the issues that might occur when testing using a different browser or different viewport such as a phone, much less fix those issues. If cross-browser testing was not included as a step in the project and the browsers in which the site must work were not specified in the requirements, then it’s anyone’s guess as to which browsers the site will work in.

Deadlines, or the timeline of the project, is one of the three levers that can be controlled within a software development project. The other two levers are cost and features. The adage is that a given

project can choose two of the three, meaning that if the project needs to be done quickly and with many features, then costs will increase. Likewise, if a project needs many features but low costs, then completing the project will take longer. Finally, if costs must be kept as low as possible while still meeting the deadline, then features are the first thing to be sacrificed.

Figure 1-5 illustrates the concept of the software development triangle.

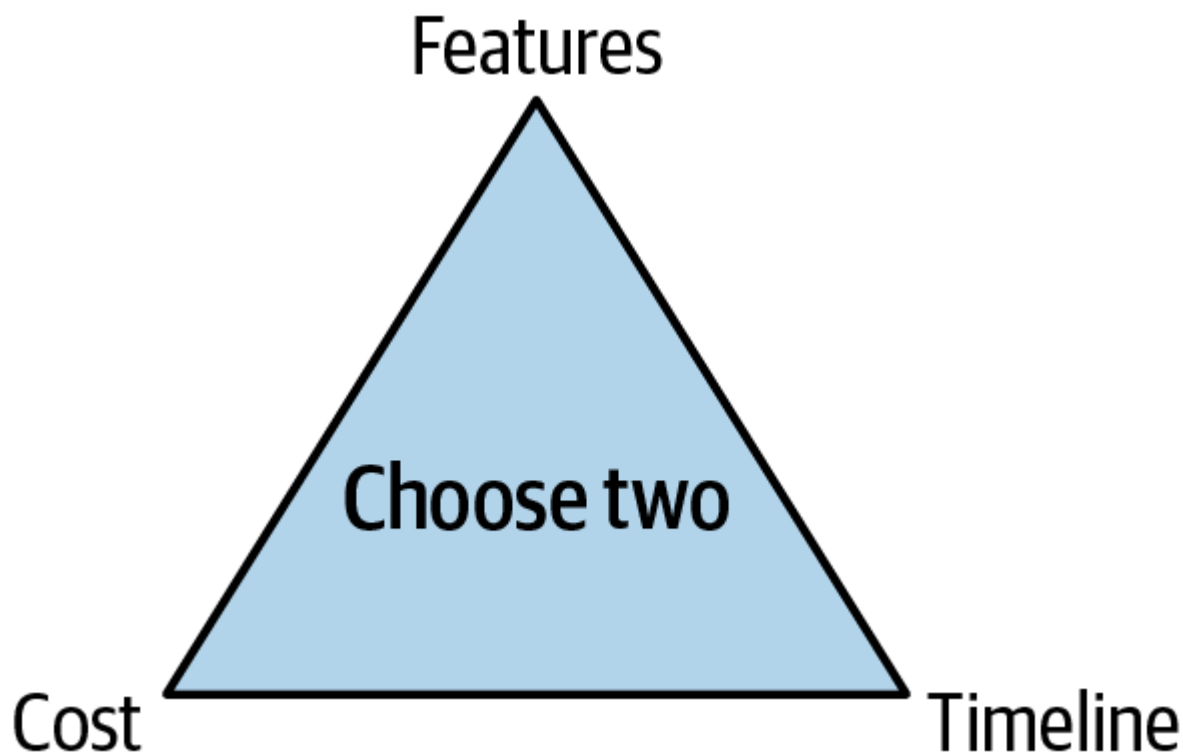


Figure 1-5. Choose two of the three elements at any one time

The next problem I'll address is the handoff between development and QA.

Operating in a Darkroom

Somewhere between development and testing lies an all-too-often awkward handoff between those who developed the software and those who are now charged with deploying, operating, and supporting the software in its production environment: the operations team. The operations team may be known by many names, including network administrators, system administrators, or engineering (site reliability engineer [SRE], production engineers, and the like), among others.

The operations team needs to take software that may never have been tested on a computing environment like the one in production and run it according to the service-level agreement (SLA) needed by the organization. That software may have only been tested on developer workstations and then a small quality assurance (QA) environment. The QA environment may have an entirely different configuration—for example, it may be lacking a load balancer, may be deployed in a different region, and may be significantly less busy than its production counterpart. Nevertheless, the software is deployed into production, and the operations team needs to support it.

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

Consider this scenario: up until the moment that the software was deployed, everything worked well. There was virtually no latency for any requests, and even when all of the developers were working on the site, response times were unremarkable. Unnoticed was that the developers were using a server that was physically located on the same local area network (LAN) as they were and that the data being used by the application came from a nonproduction replica that rarely receives any requests.

When the software was deployed to production by the operations team, the site was instantly underperforming to the point of being unusable. Users logging in were unable to continue because sessions were spread across multiple servers instead of just the one that the developers were using during the entire development lifecycle. And then you have the security problem.

Security as an Afterthought

A “ship at any cost” mentality can exist in some organizations along with a “minimally viable product” (MVP) attitude. While in theory such a development paradigm might work, the assumption is that there will be time allocated to circle around and fix the issues that made the software “minimally viable” in the first place. That time rarely exists.

When deadlines loom, security seems to be the first requirement to be sacrificed, assuming security was thought of at all. Much like math, security is hard. Security analysts need to be right every time, while an attacker only needs to be right once.

Too often, the data security department within an organization is seen as the department that says “no.” Whether you’re talking about a request for a new application, a firewall change, or relaxing rules on database access, the people tasked with maintaining security necessarily lean toward saying no when a change request comes through.

The inherent problem both with operations and data security is that they are invisible until something goes wrong. In the case of data security, much time is spent responding to compliance audits that seemingly add very little value to day-to-day security for many organizations. Make no mistake, legal and regulatory compliance is essential, but regulations often lag reality, meaning that the regulations capture compliance against yesterday’s vulnerability while the attackers are using the latest zero-day.

In the context of DevSecOps, security integration is necessary early so that firewall changes or noncompliant methods of accessing and storing data are never even considered. Without security integration, a developer might use unencrypted passwords or store credentials in the source code management system, potentially exposing them to individuals who are not authorized to view the data.

This section addressed many of the issues associated with software development, some of which are solved with DevOps and DevSecOps. Next, I’ll dive into how your organization’s culture can determine your success with DevSecOps.

Culture First

Organizational culture is the primary factor that determines whether DevSecOps will be successful. A control-oriented, top-down organization will struggle with the changes necessary to truly implement DevSecOps. Such an organization may use technology that feels like DevSecOps, but the cultural shift toward cross-team pollination will prevent true success.

A certain appreciation for the importance of cultural fit is not possible unless and until you've experienced trying to implement Agile-like practices in a rigid control-oriented organization. In such an organization, the best solution is less important than subordination and maintaining separation to keep control at the top. Without that experience, it might be possible to believe that culture plays no role in DevSecOps success.

Of course, anarchy and chaos isn't the goal of DevSecOps. Instead, DevSecOps facilitates a problem-solving approach even if the solution comes from someone in a different department. Some may believe that DevSecOps thrives when used with a startup mentality (historically a much more flexible culture), but the movement is much more nuanced than that.

A startup mentality implies both competitiveness and innovation, breaking new ground without regard to hierarchy. The founder of a startup frequently works alongside employees as their peer, possibly mentor, to drive the product forward. In a startup, job titles are less important than ensuring that the work is accomplished.

Within DevSecOps, people work together across job functions, using their skills where needed. Like a startup, the team is transparent about their work, focusing on the end goal of accomplishing useful work. In such an environment, potential problems can be identified and addressed early, well before that problem becomes visible.

DEVOPS WITHOUT SEC

Before DevSecOps, there was DevOps (Development and Operations). However, the realization soon occurred that development and operations cannot be successful without meaningful integration of security best practices. By integrating security discussions at project inception, security can become pervasive but not invasive.

DevOps can exist and be helpful even if the organization is not ready to fully integrate security. However, the same problems that led to the DevOps movement, where problems with a deployment are not found until too late, can happen because of the (necessary) security controls in place within the production or live environment. When that occurs, momentum builds toward DevSecOps as a cultural change.

The next section looks at the core of DevOps, which is an emphasis on processes versus the tools used to implement those processes.

Processes over Tools

DevOps and DevSecOps are more about processes than the tools used to implement those processes. Without the cultural fit and changes to process, the tooling used in DevSecOps often gets in the way of progress and sometimes slows development down. Even if an organization isn't ready to make the cultural changes needed for true DevSecOps, some benefit is possible by using a few of the best practices underlying DevSecOps. Let's explore a few of those now, starting with knowing how to recognize the talent who will embrace DevSecOps.

Promoting the Right Skills

Management buy-in and visible commitment to DevSecOps processes is the absolute final arbiter over whether DevSecOps will be successful. Merely having teams talk to one another is a first step, though likely more symbolic than productive. Managers can't expect to bring people together who have interests that sometimes clash and expect magic to happen.

The processes involved in finding value with DevSecOps require varied skill sets that cut across functional areas. For example, a developer that also deploys their own clusters and can articulate the difference between DNS and DHCP is a candidate for a DevSecOps pilot program within an organization. Therefore, identifying the employees who have cross-functional experience is the true first step. Those individuals can be used to champion efforts around DevSecOps.

Identifying eclectic skills and then enabling employees with those skills to cross functional boundaries is the first step of the process and illustrates the importance of management and executive buy-in for DevSecOps. Developers will need access to, or at least visibility into, server and network areas that may have been solely under the purview of Operations. Operations and Security staff will need to have substantive early input within the project lifecycle so that they can provide feedback to improve downstream processes. For example, a change for a project in development will increase disk utilization immensely. However, with a slight change to the project, utilization can be shifted onto a different system. The opportunity to implement that change would only be available early in the development process, which is why having Operations staff substantively involved in every project is important.

DevSecOps as Process

The process of DevSecOps brings people from different functional areas together. Once together, the goal is to produce better software—software that meets requirements and is delivered rapidly and accurately. The process of delivering this software can, and frequently does, involve tooling. Let's explore some of the processes in this next section.

Hammers and screwdrivers

Tools are essential to complete some jobs efficiently. A roofer used a nail gun attached to a compressed air tank to attach shingles to my roof. That same job could have been done using a hammer but would have been much more difficult to accomplish with a screwdriver. Sure, the contractor could've used the handle of the screwdriver to drive the nails through, but doing so would have been slow and inefficient and would have resulted in nails being bent and shingles being damaged. Put me on the roof trying to handle the nail gun, and there would have been at least one trip to the emergency room.

DevSecOps is similar. Just as properly roofing a building takes a combination of skilled workers and tools, DevSecOps requires tools and the know-how to use the tools properly. Just as a powerful nail gun is the right tool when used by a qualified person, DevSecOps tooling can provide huge efficiency gains when used by the right people.

The tool should help complete the job, but the tool does not define the job.

Repeatability

DevSecOps focuses on building repeatable processes, which then facilitates automation. Or perhaps it is the other way around. Automation facilitates repeatable processes. Yes, both are true. Automating the creation of environments and the deployment of code enables those processes to be repeated, time and again, with the same result. Automated testing relieves the burden of needing to manually test and retest the same areas of code, even after changes or bug fixes have been implemented.

When implementing processes and tools to assist in repeatability, an “as Code” paradigm comes to the foreground in organizations practicing DevSecOps. “Infrastructure as Code,” “Configuration as Code,” “Everything as Code” are terms that all refer to the same concept: manage as much as possible using source code management tools and processes.

Most servers use text files or text-like files to store configuration elements. These text files can be stored in a source code management tool such as Git. Doing so enables versioning of configuration changes. For example, other administrators can look back through the commit history and see that I used an underscore in a DNS hostname once and took thousands of domains offline. At least that repository is not publicly available, so no one will find my mistake. In seriousness, versioning configuration changes makes for rapid recovery if there is an issue caused by a configuration change. Source code management practices for server configurations also facilitate versioning, meaning that developers can deploy a certain set of configurations to re-create a bug being reported using the same environment.

The same set of configurations with the same versions of software makes software deployment repeatable. Repeatable deployment is directly connected to continuous integration/continuous deployment (CI/CD) scenarios, where code is automatically tested and promoted through a series of environments before being promoted to the production environment. An administrator changes a configuration element for a service, commits the configuration file, and pushes the change to the

remote repository where the change is noticed and deployment is automatically started to the appropriate servers.

NOTE

I'm purposefully ignoring the numerous formats that are used to store configurations such as Yet Another Markup Language (YAML), INI file structure, Extensible Markup Language (XML), JavaScript Object Notation, brew scripts, m4 commands, and any other structure that can be edited with a text editor like Vim. For the purposes of this book, and unless doing so would cause undue confusion, you'll see all of these formats simply referred to as text files. Here's an example of YAML:

```
- name: add docker apt key
  apt_key:
    url: https://download.docker.com/linux/debian/gpg
    state: present

- name: add docker repo
  apt_repository:
    repo: deb [arch=amd64] \
https://download.docker.com/linux/debian stretch stable
    state: present
```

Visibility

DevSecOps also serves to enable visibility throughout the development process. Not only is there frequent visibility through an Agile ceremony like Daily Standup, but there's also visibility through the tooling that deploys code automatically to environments on demand. Members of a DevSecOps team can see exactly which code and configurations exist in which environments and can deploy new environments as needed.

Reliability, speed, and scale

Repeatability and visibility lead to reliability. Code and environments can be deployed consistently, time and again, in the same way. If there is an error during deployment, that error is found immediately because of the visibility inherent in the deployment tools and processes. With reliability then comes speed, or the ability to quickly react to changing needs. That change may be a need to scale up or down based on demand, which is possible and no longer difficult because of the repeatable and reliable processes involved in deployment.

Microservices and architectural features

Though not directly required for DevSecOps, the use of microservices can serve as an enabler of speed and scale. With microservices, small functional areas of code are identified and separated such that those functional areas can stand on their own, providing a consistent application programming interface (API) to other services within the architecture. The API is frequently expressed through an HTTP web

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

service. Being standalone, microservices can be developed and deployed separately from other services or functional areas, thereby further increasing overall speed and development momentum.

This section looked at some of the processes involved in DevOps and DevSecOps. The next section expands on the SDLC shown earlier in the chapter, incorporating the ideas behind the processes to create an expanded SDLC for DevSecOps.

The DevSecOps SDLC

By this point, hopefully you have a feel for some of the problems inherent in software development; even relatively new methods of development like Agile foster a silo mentality. Instead of the four-phase model shown in [Figure 1-2](#), an eight-phase model has been created. This model incorporates planning, development, and testing along with other tasks and is shown in [Figure 1-6](#).

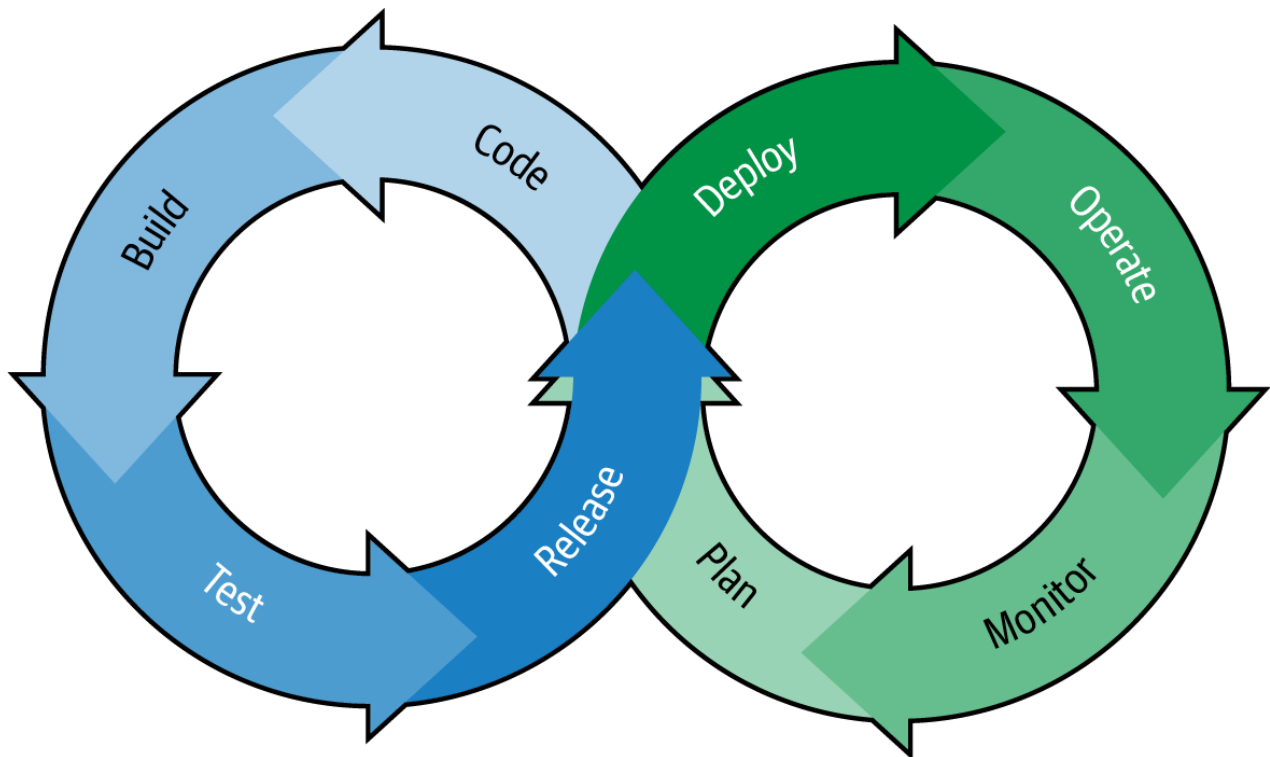


Figure 1-6. Creating a new SDLC for DevOps

The primary advantage to the DevOps SDLC is that it more closely reflects what actually happens for software development. Much more time is spent coding and testing the software than planning to code and test the software, but the interim “build” step reflects the assembly stage where the various pieces that comprise a modern application are connected to one another. Likewise, the “release” step reflects the need for multiple components along with potential approval gates through which the software must pass to begin deployment. Not captured in the SDLCs covered in this chapter is the need to both operate and monitor the software after it goes live. Without the “operate” and “monitor” phases, the Operations team becomes invisible again.

This document was truncated here because it was created in the Evaluation Mode.

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.